What is Operating system? Explain its functions in brief.

An **Operating System (OS)** is a specialized piece of software that acts as an intermediary between the computer hardware and the user. Its primary goal is to provide an environment in which a user can execute programs in a convenient and efficient manner.

Without an OS, you would have to write complex machine code just to perform simple tasks like saving a file or printing a document.

Key Functions of an Operating System

The functions of an OS can be broadly categorized into the following areas:

1. Process Management

The OS manages the execution of multiple programs simultaneously. It decides which process gets the CPU, for how long, and handles the transition between states (Ready, Running, Waiting).

Key Tasks: Creating/deleting processes, CPU scheduling, and synchronization.

2. Memory Management

The OS keeps track of every memory location, whether it is allocated to some process or it is free. It decides which process will get memory when and how much.

Key Tasks: Allocation and de-allocation of RAM, and managing "Virtual Memory" when RAM is full.

3. File Management

A computer stores information on various physical media (HDDs, SSDs, Magnetic Tapes). The OS provides a uniform logical view of information storage by organizing data into **Files** and **Directories** (Folders).

Key Tasks: Creating, deleting, and searching files; managing file permissions (Security).

4. Device Management

The OS manages device communication via their respective drivers. It keeps track of all devices (Printers, Keyboards, Monitors) and decides which process gets access to the device and for how long.

Key Tasks: Input/Output (I/O) scheduling and buffering.

© Ramit Pariyar

5. Security and Error Detection

- ✓ **Security:** Prevents unauthorized access to programs and data by using passwords and similar other techniques.
- ✓ **Error Detection:** Constantly monitors the system to detect errors (like bad sectors on a hard disk or software crashes) to prevent system failure.

What is producer consumer problem? Write the solution for producer consumer problem.

The **Producer-Consumer Problem** (also known as the **Bounded-Buffer Problem**) is a classic multi-process synchronization challenge in Operating Systems.

The Problem Statement

There are two processes, the **Producer** and the **Consumer**, who share a common, fixed-size **Buffer** (queue).

- ✓ **The Producer:** Generates data and puts it into the buffer.
- ✓ **The Consumer:** Takes data out of the buffer to process it.

The Constraints (The Real Problem):

- ✓ **Buffer Overflow:** The Producer must not add data if the buffer is full.
- ✓ **Buffer Underflow:** The Consumer must not remove data if the buffer is empty.
- ✓ **Race Condition:** Both must not access the buffer at the same time (Mutual Exclusion).

The Solution (Using Semaphores)

The most common solution uses three **Semaphores** to coordinate the work:

- ✓ **Mutex (m):** A binary semaphore (0 or 1) used to ensure that only one process accesses the buffer at a time.
- ✓ **Empty (e):** A counting semaphore that keeps track of the empty slots in the buffer (Initially set to N, the buffer size).
- ✓ **Full (f):** A counting semaphore that keeps track of filled slots (Initially set to 0).

© Ramit Pariyar

Define the term paging. Explain different memory allocation strategies.

Paging is a memory management scheme that eliminates the need for contiguous allocation of physical memory. This scheme permits the physical address space of a process to be non-contiguous.

- ✓ Logic: The logical memory (process) is broken into fixed-size blocks called Pages.
- ✓ Physical: The physical memory (RAM) is broken into fixed-size blocks called Frames.
- ✓ Mapping: A Page Table is used to keep track of which page of a process is stored in which frame of the RAM.

2. Memory Allocation Strategies

In an Operating System, memory allocation is the process of assigning portions of computer memory to programs. These strategies determine how the OS finds a "hole" (free space) in memory for a new process.

The three most common strategies are:

A. First Fit

The OS scans the memory from the beginning and allocates the **first hole** that is big enough to hold the process.

- ✓ **Advantage:** It is the fastest strategy because it stops searching as soon as it finds a fit.
- ✓ **Disadvantage:** It can lead to "Internal Fragmentation" at the beginning of the memory.

B. Best Fit

The OS scans the entire list of free holes and allocates the **smallest hole** that is big enough to hold the process.

- ✓ **Advantage:** It leaves the smallest leftover space, making use of memory more "tightly."
- ✓ **Disadvantage:** It is slow (must search the entire list) and can leave many tiny, useless holes that are too small for any process.

© Ramit Pariyar

C. Worst Fit

The OS scans the entire list and allocates the **largest available hole** for the process.

- ✓ **Advantage:** The leftover space is usually large enough to be used by another process.
- ✓ **Disadvantage:** It is the least efficient in terms of memory utilization and is also slow because it requires searching the whole list.

Differentiate between internal and external fragmentation.

Basis	Internal Fragmentation	External Fragmentation
Definition	Wastage of memory inside allocated block	Wastage of memory outside allocated blocks
Cause	Fixed-size memory allocation	Variable-size memory allocation
Memory Structure	Memory is divided into equal blocks	Memory is allocated dynamically
Wasted Space	Inside the allocated partition	Between allocated partitions
Occurrence	Paging, fixed partitioning	Segmentation, dynamic partitioning
Utilization	Unused space cannot be reused	Free space exists but not usable
Solution	Reduce block size	Compaction, paging
Complexity	Simple to manage	More complex to manage

Explain deadlock in OS. Explain necessary conditions for deadlock in OS.

1. Definition of Deadlock

A **Deadlock** is a specific situation in an Operating System where a set of processes are blocked because each process is holding a resource and waiting for another

resource held by some other process in the group. As a result, none of the processes can move forward, and the system becomes stuck.

2. Necessary Conditions for Deadlock (Coffman Conditions)

For a deadlock to occur, the following four conditions must hold **simultaneously**. If any one of these conditions is prevented, a deadlock cannot happen.

I. Mutual Exclusion

At least one resource must be held in a **non-sharable** mode. This means only one process can use the resource at any given time. If another process requests that resource, the requesting process must wait until the resource has been released.

Example: A Printer or a Magnetic Tape drive.

II. Hold and Wait

A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.

Example: Process $P1$ holds the Scanner but is waiting for the Printer, which is currently held by Process $P2$.

III. No Preemption

Resources cannot be forcibly taken away from a process. A resource can only be released voluntarily by the process holding it, after that process has completed its task.

Example: The OS cannot "kill" a process just to take its RAM and give it to another unless designed to do so.

IV. Circular Wait

A set of processes must exist P0, P1, P2, …., Pn such that P0 is waiting for a resource held by P1, P1 is waiting for a resource held by P2, and Pn is waiting for a resource held by P0. This forms a circular chain of dependency.

What is deadlock detection? Explain banker's algorithm with example.

Deadlock Detection is a strategy where the Operating System does not try to prevent or avoid deadlocks. Instead, it allows the system to enter a deadlock state, periodically runs an algorithm to check if a deadlock has occurred, and then takes action to recover from it.

© Ramit Pariyar

How it works: The OS maintains a **Resource Allocation Graph (RAG)** or structures like "Allocation" and "Request" matrices. If the algorithm finds a **cycle** in the graph (for single-instance resources), it confirms a deadlock.

Recovery: Once detected, the OS recovers by either **aborting processes** (killing them) or **preempting resources** (taking resources away from one process and giving them to another).

2. Banker's Algorithm (Deadlock Avoidance)

The **Banker's Algorithm** is a deadlock avoidance strategy. It is named so because it mimics a bank's way of ensuring it never runs out of cash to satisfy all its customers.

Before allocating a resource, the OS simulates the allocation. If the resulting state is **"Safe"** (meaning there is a sequence where everyone can eventually finish), the resource is granted. If not, the process must wait.

Data Structures Used:

- ✓ **Available:** A 1D array showing the number of available resources of each type.
- ✓ **Max:** A 2D array showing the maximum demand of each process.
- ✓ **Allocation:** A 2D array showing resources currently held by each process.
- ✓ **Need:** A 2D array showing remaining resources needed (**Need = Max – Allocation**).

3. Example of Banker's Algorithm

Assume we have 3 Processes (P0, P1, P2) and 3 Resource types (A, B, C).

Total Resources in System: A=10, B=5, C=7

Step 1: Given State

Process	Allocation (A B C)	Max (A B C)	Available (A B C)
P0	0 1 0	7 5 3	**3 3 2**
P1	2 0 0	3 2 2	
P2	3 0 2	9 0 2	

Step 2: Calculate "Need" Matrix (Need = Max - Allocation)

P0 Need: (7-0, 5-1, 3-0) = 7, 4, 3

P1 Need: (3-2, 2-0, 2-0) = 1, 2, 2

P2 Need: (9-3, 0-0, 2-2) = 6, 0, 0

Step 3: Finding the Safe Sequence

We check which process's **Need** is ≤ **Available**.

Check P0: Need (7,4,3) is NOT ≤ Available (3,3,2). **[P0 waits]**

Check P1: Need (1,2,2) IS ≤ Available (3,3,2). **[P1 Runs]**

After P1 finishes, it releases its allocation: New Available = 3,3,2 + 2,0,0 = 5,3,2.

Check P2: Need (6,0,0) is NOT ≤ Available (5,3,2). **[P2 waits]**

Check P0 again: Need (7,4,3) is NOT ≤ Available (5,3,2). **[P0 waits]**

Wait! In this specific example, if no other process can finish, the system is in an **Unsafe State**. However, if we had more resources:

If P1 finished and the new available was enough for P0, the sequence would be <P1, P0, P2>.

Explain different file allocation techniques in Os.

In an Operating System, File Allocation Techniques determine how files are stored in the physical sectors of a hard disk. Since the disk is divided into fixed-size blocks, the OS must decide which blocks to assign to a file to maximize speed and minimize wasted space.

The three primary techniques are:

1. Contiguous Allocation

In this method, each file occupies a **consecutive** set of blocks on the disk. For example, if a file starts at block b and has a size of n blocks, it will occupy blocks b, b+1, b+2,….., b+(n-1).

Advantages:

- ✓ High Performance: Very fast for sequential access because the disk head doesn't have to move much.
- ✓ Easy to implement: The OS only needs to store the *Starting Block* and the *Length*.

Disadvantages:

- ✓ External Fragmentation: As files are deleted, small "holes" of free space are left that might be too small for new files.
- ✓ File Growth: It is difficult to increase the size of a file if the next block is already occupied by another file.

2. Linked Allocation

In this method, each file is a **linked list** of disk blocks. The blocks may be scattered anywhere on the disk. Each block contains a pointer to the next block.

Advantages:

- ✓ No External Fragmentation: Any free block can be used to store part of a file.
- ✓ Flexible Size: Files can grow easily as long as there are free blocks available anywhere.

Disadvantages:

- ✓ Slow Random Access: To find block 10, the OS must read blocks 1 through 9 first to get the pointers.
- ✓ Reliability: If one pointer is corrupted or lost, the rest of the file becomes unreadable.
- ✓ Space Overhead: A small portion of every block is wasted to store the pointer.

3. Indexed Allocation

To solve the problems of both contiguous and linked allocation, **Indexed Allocation** brings all pointers together into one location called the **Index Block**. Each file has its own index block, which contains the addresses of all its disk blocks.

Advantages:

- ✓ Efficient Random Access: The OS can jump directly to any block by looking at the index.
- ✓ No External Fragmentation: Blocks can be scattered across the disk.

© Ramit Pariyar

Disadvantages:

- ✓ Index Block Overhead: For very small files, a whole block is wasted just to store a few pointers.
- ✓ Pointer Limit: If a file is extremely large, one index block might not be enough to hold all the pointers (this is solved using *multilevel indexing*).

Define cryptography. Explain different security attacks

Cryptography is the science and practice of securing information by transforming it into an unreadable format. Its primary goal is to ensure that only the intended recipient can read and process the data.

- ✓ Encryption: The process of converting plain text (readable) into ciphertext (unreadable).
- ✓ Decryption: The process of converting ciphertext back into plain text.

In an Operating System or Network environment, cryptography provides four essential services: Confidentiality (privacy), Integrity (no tampering), Authentication (identity), and Non-repudiation (cannot deny sending).

Security Attacks

Security attacks are generally divided into two main categories: Passive Attacks and Active Attacks.

1. Passive Attacks

A passive attack is one where the attacker only monitors or eavesdrops on the data being transmitted. They do not alter the data. These are very difficult to detect because the system continues to function normally.

- ✓ Release of Message Content: An attacker reads a private email or file being sent over a network.
- ✓ Traffic Analysis: Even if the message is encrypted, the attacker observes the pattern, frequency, and location of communication to guess the nature of the data.

2. Active Attacks

An active attack involves the attacker modifying the data stream or creating a false stream. These are easier to detect but harder to prevent.

© Ramit Pariyar

- ✓ Masquerade: The attacker pretends to be a different person (e.g., logging in with stolen credentials).
- ✓ Replay: The attacker captures a valid data transmission (like a login request) and re-sends it later to gain unauthorized access.
- ✓ Modification of Messages: The attacker intercepts a message and changes it (e.g., changing "Send 10" to "Send 1000").
- ✓ Denial of Service (DoS): The attacker floods a system with fake requests, making it crash or become so slow that legitimate users cannot access it.

Write short notes:

A. Race condition

In Operating Systems, a Race Condition is an undesirable situation that occurs when a device or system attempts to perform two or more operations at the same time, but because of the nature of the device or system, the operations must be done in the proper sequence to be done correctly.

In simpler terms, the output depends on the "race" between processes to see which one finishes first.

1. How it Happens

A race condition occurs when:

- ✓ Two or more processes access and manipulate the same data (shared resource) concurrently.
- ✓ The final value of the data depends on the order of execution (which process runs first).

2. Practical Example: Bank Balance

Imagine a shared bank account with a balance of **100**.

- ✓ Process A (ATM Withdrawal): Wants to withdraw 50.
- ✓ Process B (Online Deposit): Wants to deposit 50.

The Error Scenario:

- ✓ Process A reads the balance (100).
- ✓ Process B reads the balance (100) before Process A finishes.
- ✓ Process A subtracts 50 and writes the new balance: 50.
- ✓ Process B adds 50 to its initial reading (100) and writes the balance: 150.

Instead of the correct balance (100 - 50 + 50 = 100), the final balance is **150** because Process B overwrote Process A's work. This is a Race Condition.

3. The Critical Section

The part of the program where the shared memory is accessed is called the Critical Section. To prevent race conditions, we must ensure that only one process is in its critical section at a time. This is called Mutual Exclusion.

4. How to Prevent Race Conditions

Technicians and developers use synchronization tools to stop processes from "racing":

B. Multi Programming

Multi programming is a technique used in modern Operating Systems to increase CPU Utilization. In a multi programming system, multiple programs (processes) are loaded into the main memory (RAM) at the same time, and the CPU executes them one after another.

1. Why do we need Multi programming?

In old systems (Uni programming), the CPU would run one program and wait if that program needed to perform an I/O operation (like waiting for a user to type or a printer to finish). During this wait, the CPU sat idle, which was a waste of expensive processing power.

In **Multi programming**:

When the currently running program waits for I/O, the OS switches the CPU to another program that is ready to run.

This ensures the CPU is almost always busy.

2. How it Works

- ✓ Several processes are kept in the Main Memory simultaneously.
- ✓ The Operating System selects one of the processes and starts executing it.
- ✓ If that process needs to wait (for a disk read, user input, etc.), the OS switches to the next process.
- ✓ Once the I/O for the first process is done, it gets back in line to use the CPU again.

C. System call.

A System Call is the programmatic way in which a computer program requests a service from the Kernel of the Operating System.

In simple terms, it acts as an interface (bridge) between a user application and the hardware. Since user programs are not allowed to access hardware directly for security reasons, they must "ask" the OS to do it for them via a System Call.

1. Why are System Calls Necessary?

Computer systems operate in two modes:

- ✓ User Mode: Where your applications (like Chrome, Word, or a Python script) run. Access is restricted.
- ✓ Kernel Mode: Where the Operating System runs. It has full access to the hardware (RAM, Disk, CPU).

When a program in User Mode needs to perform a task like "Open a File" or "Print a Document," it triggers a System Call to switch the CPU to Kernel Mode to perform the task safely.

2. The Workflow of a System Call

- ✧ Request: The user program calls a library function (e.g., `printf()` in C).
- ✧ Trap: The library function executes a special instruction (a "trap" or "interrupt") that alerts the CPU.
- ✧ Switch: The CPU switches from User Mode to Kernel Mode.

- Execution: The OS looks at the system call number, finds the corresponding function in the System Call Table, and executes it.
- Return: The CPU switches back to User Mode and returns the result to the program.

RP

© Ramit Pariyar